What is a Race Condition?

- Situation where timing of events influences program behavior and outcome
- Occurs when variable accessed and modified by multiple threads simultaneously
- Lack of proper lock mechanisms and synchronization between threads
- Attackers can abuse to apply discounts multiple times or exceed transaction limits

---

Key Concepts:

Program: Static set of instructions (like a recipe)

Process: Program in execution (dynamic)

- States: New → Ready → Running → Waiting → Terminated

Thread: Lightweight unit of execution within a process

- Shares memory and instructions with the process
- Multiple threads can serve multiple users simultaneously (parallel)
- Single thread serves users sequentially (serial)

Real World Analogy:

Two restaurant hosts taking reservations for the same table at the same time:

- Host 1 checks table (sees no "Reserved" tag) → starts booking
- Host 2 checks table (still sees no tag) → also starts booking
- First host to place tag wins, second host double-books

Same with threads: Thread 1 checks value → before it updates, Thread 2 checks same (stale) value

---

TOCTOU (Time-of-Check to Time-of-Use) Vulnerability:

Example A:

- Account: $100
- Thread 1: checks ($100) → withdraws $45
- Thread 2: checks (still sees $100 before Thread 1 updates) → withdraws $35
- Result: Both withdrawals succeed, incorrect final balance

Example B:

- Account: $75
- Thread 1: checks ($75) → withdraws $50
- Thread 2: checks (still sees $75) → withdraws $50
- Result: Second withdrawal should be declined but succeeds

Common Causes:

1. **Parallel Execution** - Multiple requests handled simultaneously without synchronization
2. **Database Operations** - Concurrent read-modify-write sequences
3. **Third-Party Libraries** - External components not designed for concurrent access

---

Application States (Critical for Race Conditions):

Money Transfer Example:

- States: Not Sent → Checking Balance → Sent
- Vulnerability window exists during "Checking Balance" state

Coupon Application Example:

- States: Not Applied → Checking Validity → Checking Constraints → Recalculating → Applied
- Vulnerability window exists during validation states before "Applied"

Key Insight: Between initial check and final update, there's a time window where multiple requests can slip through

---

Exploitation Challenge:

- Window of opportunity is very short (milliseconds)
- Need requests to reach server simultaneously
- Tools like Burp Suite Repeater needed for timing attacks

Detection:

- Understand normal system behavior (use once, rate limits, balance limits)
- Attempt to circumvent limits

- Map application states to identify potential race windows
- Use tools to send concurrent requests

Mitigation:

1. **Synchronization Mechanisms** - Locks ensure only one thread accesses shared resource at a time
2. **Atomic Operations** - Indivisible execution units that can't be interrupted
3. **Database Transactions** - Group operations so all succeed or all fail together